

Parking Lot Space Finder

I. INTRODUCTION

As urban areas grow, demand for parking spots continues to rise. Yet the availability of parking spaces remains limited. Limited and decreasing availability of land especially in urban areas and zoning regulations along with increased population and vehicle ownership makes availability of parking a scarce and increasingly scarce resource. Finding parking in cities is a difficult and costly endeavor. Searching for parking results in traffic congestion, raises fuel consumption, and increases carbon emissions as motorists drive around in search of parking. The constant search for open spots leads to time and energy wasted, and additional carbon emissions, which worsen air quality. Drivers could greatly benefit from an application that helps them find available parking spaces without having to drive around hoping that someone will vacate a spot for them as they drive by. Our proposed solution attempts to do just that. We leverage machine learning, to train a model on recognizing occupied vs unoccupied spots from overhead images of parking spots. A true working model of this system would likely include cameras positioned on street lights (or high structures) in the vicinity of a parking lot which feeds real time imagery into the model to notify in real time the availability of unoccupied parking spots to drivers in the vicinity currently searching for parking.

II. DATASET

The dataset consists of aerial views of parking lot spaces from overhead cameras along with text files containing the class labels (1 = vacant, 2 = occupied) and coordinates of the bounding boxes in each image.

The features of the dataset will consist of pixel values, image dimensions, bounding box annotations, and class labels. Preprocessing will involve converting the format of the labels of the bounding box coordinates to YOLO format and splitting the data for training, testing, and validation, while maintaining class balance. It is important to note four points about this dataset that limits its effectiveness.

Firstly, all the pictures of parking lots in this dataset are not directly overhead but slightly at an angle.

Secondly all the pictures including across the train, validation test split are roughly the same parking lot.

Thirdly all the pictures are during the day, which potentially limits the model's effectiveness at night.

Lastly all the images are from 2012, while vehicle shape, size and colour hasn't changed extensively in the last 10-15 years, the effectiveness of the model on more unique and exotic car model designs such as Tesla's Cyber Truck has not been experimented on.

III. METHODOLOGY

We used a YOLO (YOLO 11 nano) pretrained model from Ultralytics with the pretrained COCO weights to perform object detection on Google Colab. We will use YOLO because as a pre-trained model it is already trained at detecting vehicles. COCO is a vehicle detection dataset, so YOLO's pre-training directly transfers to our task of identifying if a parking space contains vehicles or if it is empty. A key feature that makes YOLO attractive to use is its ability to analyze an entire image at once and thus accurately identify an occupied and empty parking space and differentiate the two. Whereas if we were to use a simple CNN then it would crop the first parking spaces in the picture, check the parking spot and save the result and repeat this process for every picture in the dataset. Another option was the use of Faster R-CNN which would be able to achieve a higher accuracy than YOLO, but it would require more computational resources. Both the use of a simple CNN or the use of a Faster R-CNN becomes unviable due to how computationally expensive it would be and how slow it would be to train.

While YOLO is a powerful tool, our solution involved a significant amount of dataset preprocessing to make it YOLO compatible. Our code solution involved taking a combined YOLO label file containing all the bounding boxes and splitting into separate txt files, one for each image of a parking lot. Different frameworks use different coordinate systems for bounding box labels, so we also had to convert those bounding box labels for each image txt file into YOLO's normalized center format using scaled image width and height and

center of image. As mentioned before our dataset identified unoccupied vs occupied spots with a 1 and 2 respectively our implementation required that we convert these class ID's from 1 based to 0 based indexing for YOLO compatibility. YOLO also relied on the use of YAML files to configure the dataset and to let our model know where it can access image file and label file data in our stored drive and our code required this configuration.

We then split the data into a train, validation and testing set. Introducing a validation set before testing the model is an important step as it helps tune parameters and prevent overfitting, a model that keeps improving on the training set but starts performing worse on the validation set is a clear sign the model is just memorizing the training data instead of learning to classify novel instances of data points. In all, our training set consisted of 6814 images, 1701 images in the validation set and 10 images in the test set. Every image as described before came with a label file which delineated the coordinates of bounding boxes which surrounded parking spots in a parking lot image.

Training the model took 24 hours on google Colab and we used Google's T4 GPUs to do all the training. Due to GPU restrictions we had to alternate between training the model on various google accounts. Due to the length of training it was important to be able to use checkpoints which allows training to be stopped and resumed at a later date. We initially attempted to train for 150 epochs, but stopped at 104 due to timing constraints. An epoch is one complete pass of the entire training dataset through the model. Training in epochs is a powerful machine learning tool, as it allows a model to gradually tune its parameters and detect overfitting as training and validation metrics such as accuracy can be compared after each epoch. We used a batch size of 8, batch size is how many training samples the model processes at once before updating weights, a small batch size trains slower and introduces more noise but helps guard against overfitting. Our code also includes a patience=50 metric which stops the training if there is no improvement in validation performance, this is important because continuing to train with worsening performance will just degrade the model further.

Other pertinent methodological details include loading the dataset into RAM for faster training and using a workers=8 metric which allows for parallel computation, essentially processing a batch for training while training on a current batch. This helps speed up training. Details like this are important considering the GPU limitations and extended training time of training over many epochs in Google Colab.

Training used 231 layers, learning 20,054,550 parameters, (a layer exists within a neural network), the main benefit of using successive training layers is that the model more accurately identifies and learns patterns from the data by learning abstract features first and then gradually learning more specific and detailed features. Our model makes use of many different types of layer architectures but focuses mainly on convolution and its derivatives such as C3K2, which is a YOLO specific convolution layer. In fact many of the layers we used such as C2PSA and SPPF are YOLO specific modules. Convolution layers are a powerful tool in classifying images and our model makes consistent use of them and it is worth discussing. A convolution layer applies many small kernels to an image, a kernel is a small weight matrix, the kernel slides over the images and computes a feature map through a matrix convolution operation with the image matrix. The feature map shows where on an image a feature occurs. An activation function takes the matrix from the previous convolution operation and introduces non-linearity and a assessment of the features choosing which features are important enough to pass forward to the next layer. A non-linear activation function is crucial to a project like this as it allows for a holistic classification of an image as images are mostly made up of non-linear data. Finally the outputted matrix from the activation function undergoes a pooling process which creates a smaller matrix, which allows the feature map to focus on the most important details which saves on computational resources and guards against overfitting.

In addition to basic convolution our model makes use of many different modules. C2PSA layers introduce attention metrics which allow the model to improve accuracy by ignoring background noise and focusing on the pertinent objects in an image (parking spaces and vehicles). Concat, which combines multiple features maps into one by stacking them together which allows a model to use abstract features and high level features at the same time (regular convolution layers already make use of this but

YOLO's Concat module improves on this). Upsample enlarges small feature maps which allows abstract information from early layers to be combined with object recognition information from deeper layers which improves on the regular convolution process and the model's ability to detect occupied vs unoccupied spots. This was especially important for us as our model was tasked with identifying the vacancy of parking spots which were quite small features on the image.

IV. RESULTS

We begin by discussing precision which is computed by the following formula $\text{True Positives} / (\text{True Positives} + \text{False Positives})$ essentially it is a measure how often the model's predicted positives are correct. As Figure 1. Demonstrates our model's precision is extremely high, and nearly perfect across an entire range of confidence which suggests very low false detections

Next we discuss Recall which is computed by the following formula $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$. Essentially, it is a measure of how good the model is at finding the actual positive cases.

As Figure 2. Demonstrates our model's recall is extremely high across an entire range of confidence which suggests very few missed detections.

The Precision-Recall curve in Figure 3. Demonstrates excellent model performance for both vacant and occupied classes. The curve remains at the top right of the plot which indicates a simultaneous high precision and recall, which is a difficult achievement for a model.

F1 score which is computed with the following formula

$(2 * \text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$ is a powerful metric as it penalizes both false positives and false negatives a F1 of 1.0 delineates perfect detection while 0 is complete failure. At a model confidence threshold of 0.48 our model achieves a perfect F1 score of 1.0 which suggests it separates occupied vs vacant spots extremely well and that there is no major trade off between precision and recall which other models often struggle with. Figure 4. Demonstrates our models F1Confidence Curve.

The confusion matrix in Figure 5. Demonstrates that the model achieves nearly perfect classification at classifying both the vacant and

occupied classes, with 53000 vacant and 45000 occupied spots classified correctly and only a few dozen misclassifications of either class. The normalized matrix in Figure 6. converts the raw prediction counts into percentages and also demonstrates the excellent classification performance. The normalized matrix also demonstrates high level of background classification which is an unfortunate error because of the way the photos in our dataset are oriented from above and at a slight angle which means spots in the top of the photo are classified as background rather than occupied or unoccupied. An example of this is shown in Figure 7.

Box loss is a measure of how wrong the model's predicted bounding boxes compared to what are the true bounding boxes, essentially how far is the model's predicted box from the object's real location. Figure 8. Demonstrates the training box loss curve, it starts high at around 1.4 but drops very fast in the first 20 epochs and smooths out to 0.17. This shows that the model learned very quickly at how to place boxes correctly, which of course is important as our model depends on accurately placing bounding boxes before making an assessment on parking spot vacancy. Figure 9. Demonstrates the validation box loss curve it also starts high but quickly falls and smooths out, the lack of a U-shaped curve demonstrates no overfitting and thus in theory should mean it generalizes well to unseen data however our validation set's images were very similar to our training set images and thus brings the utility of our validation set into question.

Figure 10. Shows the mAP50 curve and demonstrates that the model's detection accuracy increases rapidly during the first several epochs and reaches an impressive plateau value of 0.995. This suggests a high model accuracy.

Lastly Figure 11. Shows a test set result, the model does a great job of correctly classifying spots with bounding boxes but struggles with identifying spots in the background (top) of the image as parking spots to begin with which limits the utility of the model.

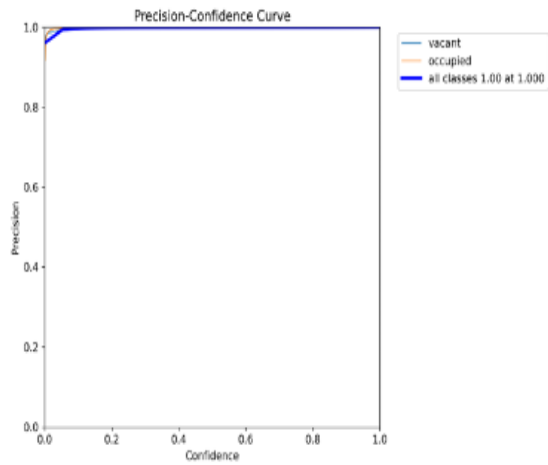


Fig 1. Precision Confidence Curve

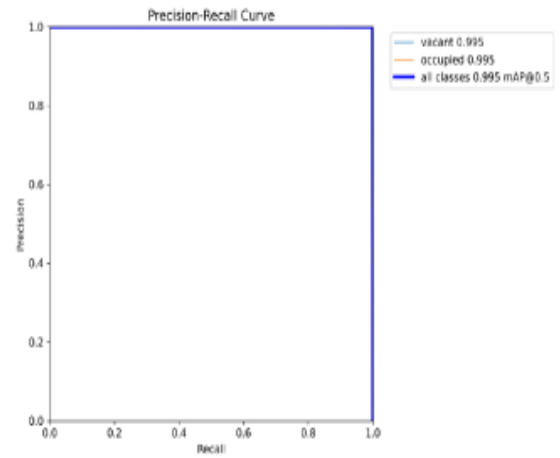


Fig 3. Precision-Recall Curve

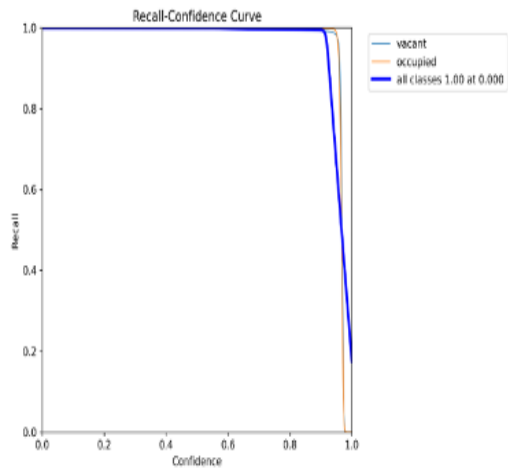


Fig 2. Recall-Confidence Curve

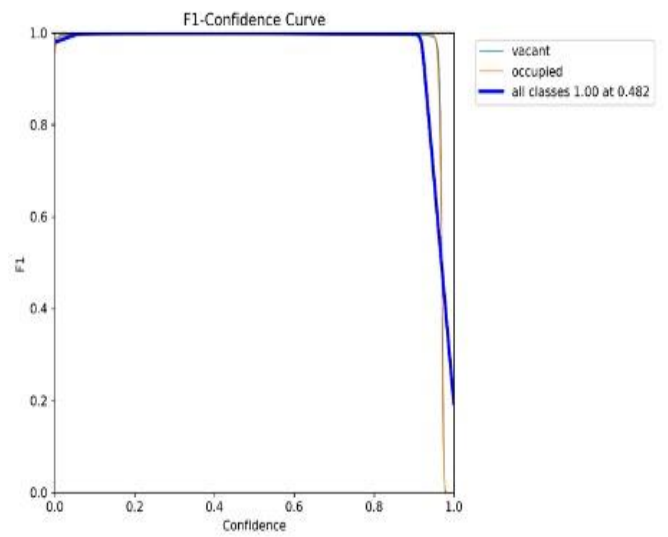


Fig 4. Confidence Curve

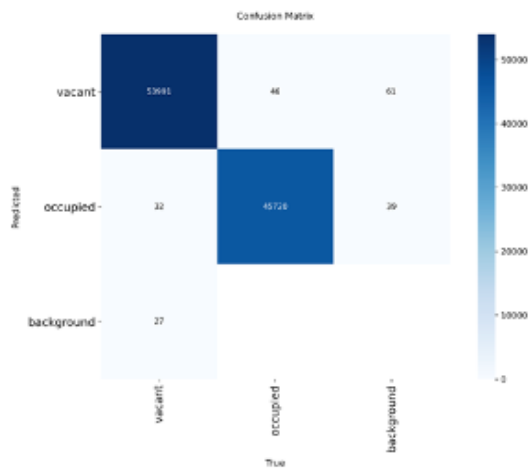


Fig 5. Confusion Matrix

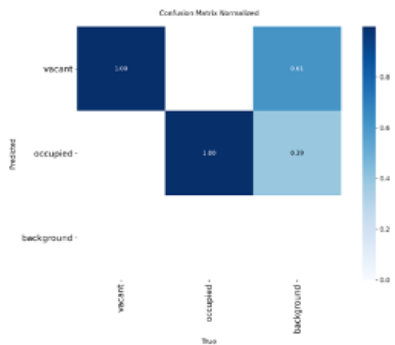


Fig 6. Confusion Matrix

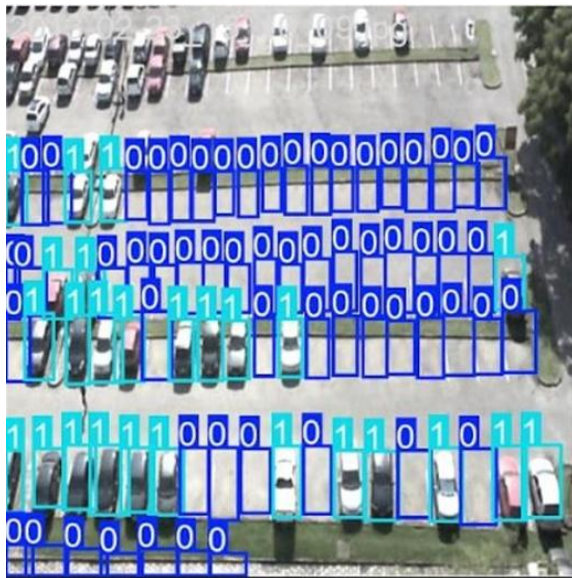


Fig 7. Background Classification Example

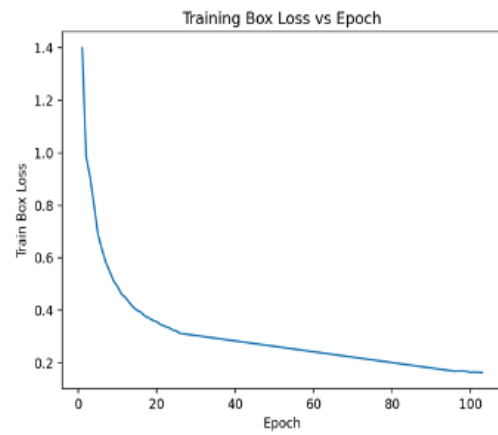


Fig 8. Training Loss vs Epoch Graph

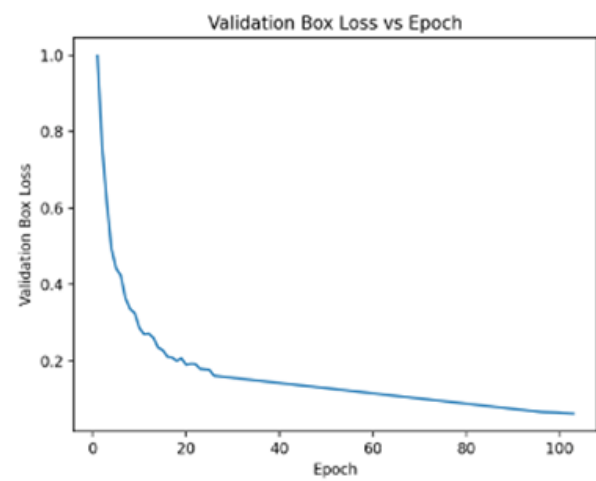


Fig 9 Validation Loss Vs Epoch

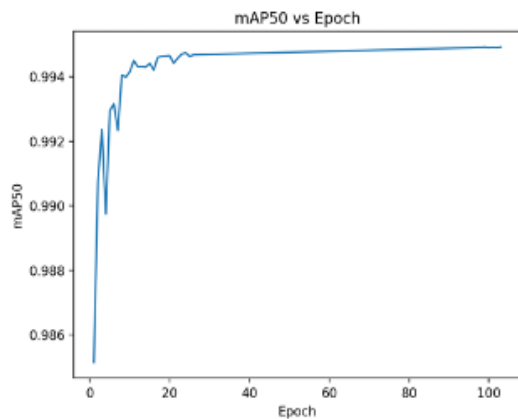


Fig 10. mAP50 Vs Epoch

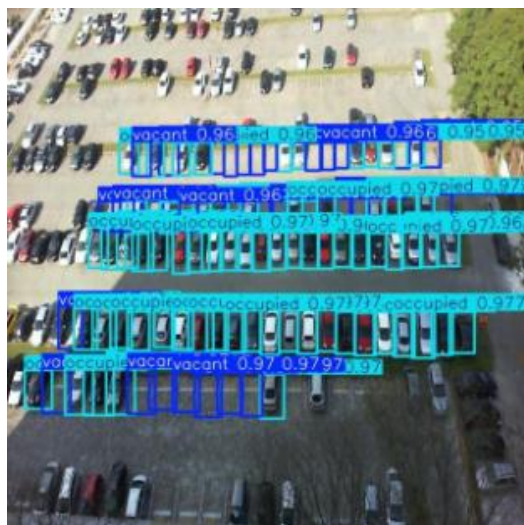


Fig 11. Classifying Bounding Boxex

V. CONCLUSION AND DISCUSSION

Our results demonstrate near perfect model performance, but the utility of the model is likely still extremely limited. Our model's successful performance metrics was, likely due to a striking lack of diversity in the dataset. Many of the images in the train, validate and test folders were very similar. Thus, while our validation metrics suggest the model is guarding against overfitting, this is likely not the case due to the

validation and training sets being so similar. Upon closer review it can be seen that many of the images are of the exact same parking lot, with the only difference being the volume of cars and vacant spots. This is a form of data leakage, and this leads to overfitting. The model is likely just memorizing and thus able to predict vacant vs occupied parking spots easily without providing a good generalizable classification scheme that is readily applicable to a variety of different parking lots.

As discussed in the dataset sections there are other issues with our dataset that could degrade or outright destroy the usefulness of our model in a real world application. The images of the parking lots in this dataset are not directly overhead but slightly at an angle, this distorts the shape and size of the parking spots in the background (top) of the image and leads to a lack of bounding box placements. This is obviously a problem as the lack of bounding boxes means the model is not even making a prediction on a spot's classification. This could be rectified by cropping images and introducing more cameras in a parking lot. The fact that all the images are of parking lots during the day in clear weather is also a problem, drivers might still need to access parking during the night or in poor weather conditions such as heavy rain or snow. Our model's effectiveness has not been tested on night time images or images of parking lots with snow or heavy rain and would likely fail in such test cases.

Despite these limitations this model is an excellent proof of concept, and demonstrates the technological feasibility of using machine learning for parking spot classification. The model can reliably distinguish between vacant and occupied spots given that the viewpoint, lighting, and parking environment remain essentially the same, a real world model, could overcome our model's limitations by using a far larger dataset, of multiple different parking lots, variations in camera angles, nighttime imagery, and images of parking lots in adverse weather

